

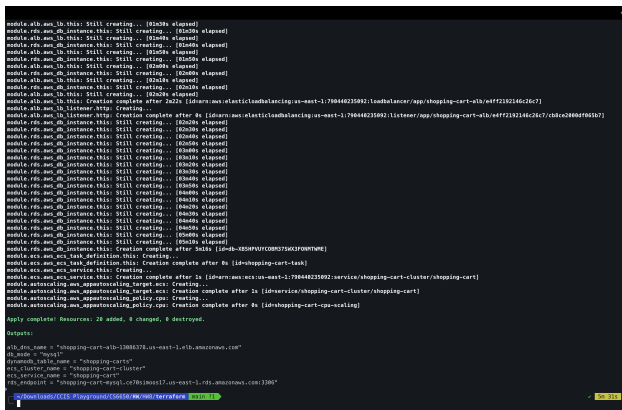
CS6650 HW8

Sai Karthikeyan Sura
March 23rd, 2026

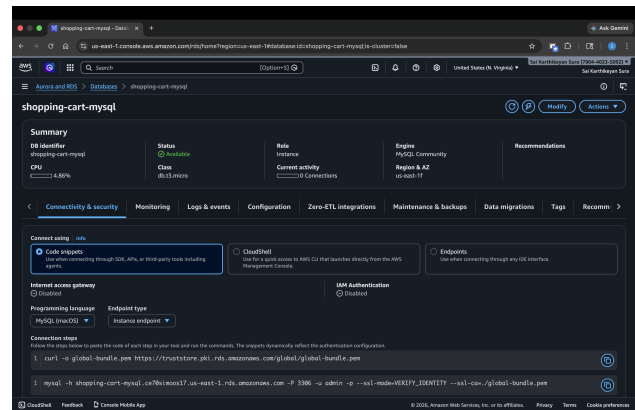
STEP I: MySQL Integration — Implementation Notes

Infrastructure & Schema Design

The HW6 ECS Fargate infrastructure was extended with an RDS MySQL 8.0 instance (db.t3.micro, free tier) via a new Terraform module (modules/rds/). A dedicated security group restricts port 3306 access exclusively to ECS tasks. The instance is configured with `skip_final_snapshot = true` and `deletion_protection = false` for teardown.



(a) Terraform apply: 20 resources, 5m31s.



(b) RDS instance: Available, db.t3.micro.

Figure 1: MySQL infrastructure deployment.

The schema uses two normalized InnoDB tables. `shopping_carts` stores cart metadata with an `idx_customer` index for purchase history queries. `cart_items` stores line items with a foreign key (ON DELETE CASCADE) and a composite unique key `uq_cart_product(cart_id, product_id)` enabling atomic INSERT ... ON DUPLICATE KEY UPDATE upserts. InnoDB row-level locking handles concurrent modifications without explicit transaction blocks for single-row operations. The normalized two-table design was chosen over a single denormalized table to enforce referential integrity at the database level and prevent orphaned items.

Connection pooling: MaxOpenConns=25, MaxIdleConns=10, ConnMaxLifetime=5min. CloudWatch confirmed a peak of only 2 concurrent connections (Fig. 4b), confirming the pool was efficient with minimal overhead for the 2 ECS tasks.

Input validation & SQL injection prevention: All SQL queries use parameterized placeholders (?) via `ExecContext` and `QueryRowContext`, ensuring user input is never interpolated into SQL strings. Request body validation uses Gin's `ShouldBindJSON` with struct tags (`binding:"required,min=1"`) to reject malformed input before it reaches the database layer.

Key Challenges

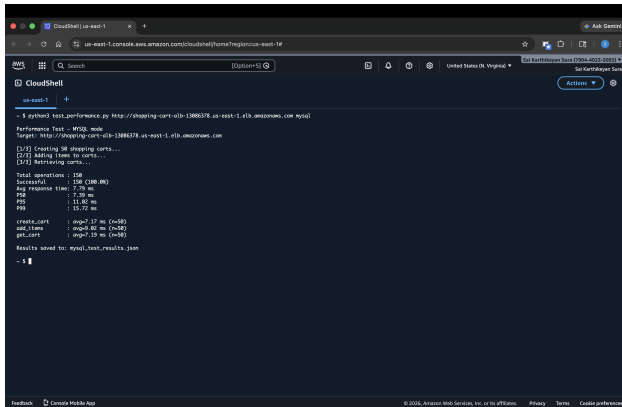
Schema migration failure: The initial implementation combined both `CREATE TABLE` statements in a single `db.Exec()` call. The MySQL driver rejected multi-statement execution. Splitting into two separate calls resolved this.

RDS readiness race condition: ECS tasks crash-looped because the Go app tried to connect before RDS was ready. Adding a retry/backoff loop (6 attempts, 10s apart) in `initMySQL()` allowed the service to wait for RDS readiness.

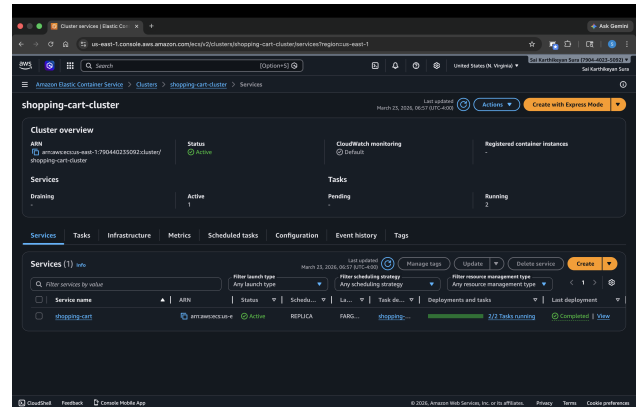
Docker image caching: After fixing the schema migration, `terraform` apply with the Docker provider did not rebuild the image due to layer caching. Running `terraform taint` also failed. Had to run `terraform destroy` and reapply to force a clean build.

Latency measurement: Initial tests from a local machine showed >50ms due to network round-trip to us-east-1. Running from AWS CloudShell (same region) revealed the true application-level latency of 7–9ms.

Performance Testing

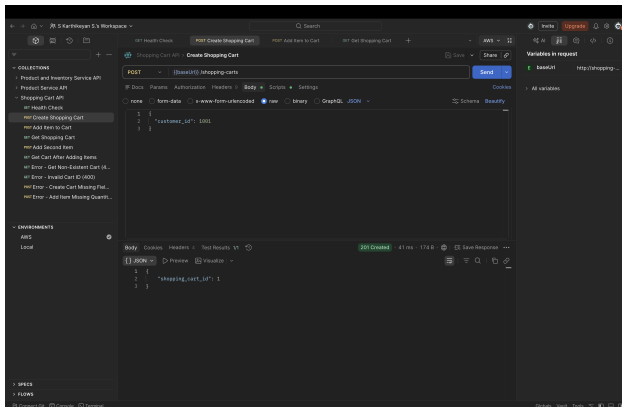


(a) 150-op test: avg 7.79ms, 100% success.

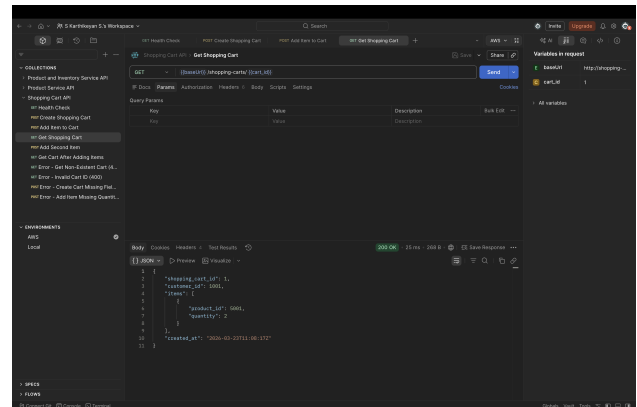


(b) ECS cluster: 2/2 Fargate tasks running.

Figure 2: MySQL performance test results and ECS deployment status.



(a) POST /shopping-carts: 201 Created, 41ms.



(b) GET /shopping-carts/1: 200 OK with items.

Figure 3: Postman API validation (MySQL mode).

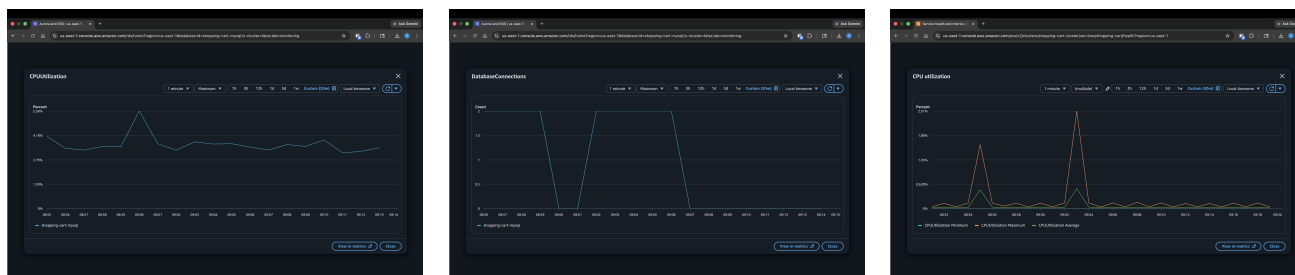
HW5 In-Memory vs. MySQL Performance Comparison

HW5's Product Service used a `sync.RWMutex`-protected Go map with zero persistence. Under Locust (50 users, 1 worker), HW5 achieved 154 RPS with 19ms median and 25ms P95. The MySQL shopping cart service under Locust (50 users, 2 workers) achieved 40.5 RPS with 38ms median and 53ms P95. The lower RPS is expected: each MySQL operation involves network round-trip to RDS plus query execution, whereas in-memory lookups are sub-millisecond. However, MySQL provides persistence across restarts, ACID guarantees, and SQL querying that the in-memory approach cannot offer. This trade-off is justified for shopping carts where data loss is unacceptable.

Learning Notes

All queries averaged well under the 50ms target. **To optimize for the 150-operation test**, I tuned the connection pool (`MaxOpenConns=25`) to prevent exhaustion and utilized `ON DUPLICATE KEY UPDATE` for atomic "add item" upserts. Exploring **new database concepts**—specifically balancing throughput against Free Tier limits via pool tuning and leveraging InnoDB's implicit row-level locking instead of manual transactions—streamlined the code significantly. Next time, I would implement database retry/backoff logic from the outset and run performance tests directly from CloudShell to avoid debugging inflated local network latency.

CloudWatch Monitoring



(a) RDS CPU: peaked 5.58%.

(b) DB connections: max 2.

(c) ECS CPU: spikes to 2.5%.

Figure 4: CloudWatch metrics during MySQL testing.

STEP II: DynamoDB Integration — Implementation Notes

Table Design & Partition Key Strategy

A single DynamoDB table with `cart_id` (UUID String) as partition key, no sort key. Items are embedded as a list attribute within the cart document. UUID keys ensure even distribution across partitions at any scale. On-demand billing (`PAY_PER_REQUEST`) eliminates capacity planning. No secondary indexes are needed since all access is by `cart_id`. The UUID partition key was chosen from the start based on DynamoDB best practices for high-cardinality keys; no changes were needed during implementation, and no hot partition issues were observed during testing.

Compared to MySQL's normalized two-table schema, DynamoDB shifts referential integrity to application code but eliminates connection pooling and JOINS. The key trade-off: add-item requires a read-modify-write cycle (`GetItem` + `PutItem`) unlike MySQL's single atomic upsert.

DynamoDB operations mapping: `PutItem` handles cart creation (write full document). `GetItem` handles cart retrieval (single partition key lookup). For adding items, `GetItem` + client-side list modification + `PutItem` handles the update — `Query` is not needed since there is no sort key and the embedded items list is returned with the cart document in a single read.

Attribute value formatting: DynamoDB's typed attribute value system (e.g., `{S: "value"}`, `{N: "123"}`, `{L: [...]}`) requires explicit marshaling between Go structs and DynamoDB's wire format. The AWS SDK's `attributevalue.MarshalMap` and `UnmarshalMap` handle this transparently, mapping Go struct fields to DynamoDB attributes via `dynamodbav` struct tags. This means application data structures (`DynamoCart`, `CartItem`) remain clean Go types without DynamoDB-specific formatting logic leaking into handlers.

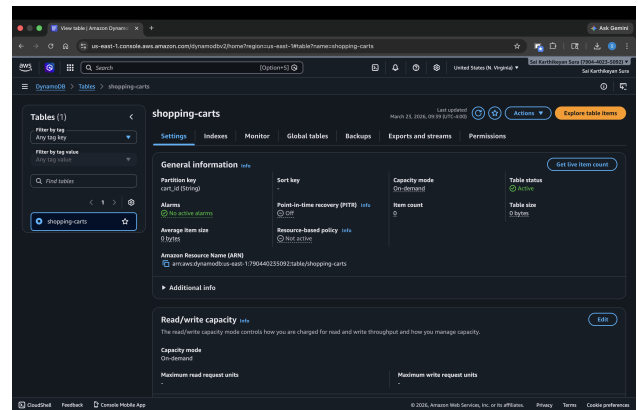
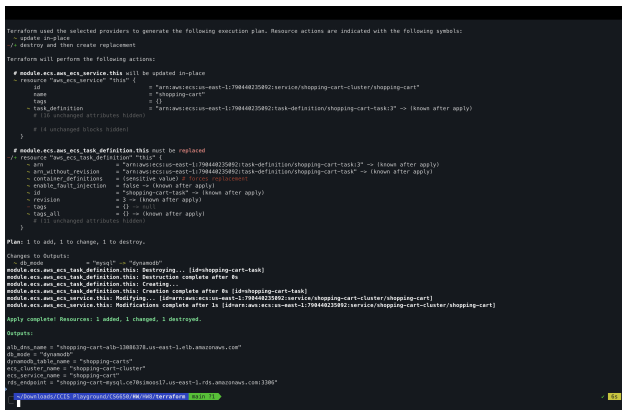


Figure 5: DynamoDB deployment and table configuration.

Eventual Consistency Investigation

Three test scenarios (20 trials each) showed 0 stale reads: read-after-write (avg 10.36ms), add-then-read, and rapid sequential updates (30 items to 1 cart, all 30 present). This is because `GetItem` uses strongly consistent reads by default. However, in production with concurrent multi-client writes to the same cart, the read-modify-write pattern could cause lost updates without conditional writes or DynamoDB transactions. To handle consistency gracefully, applications could use `ConditionExpression` with a version counter for optimistic locking, or use `DynamoDBTransactWriteItems` to atomically check-and-update the cart document.

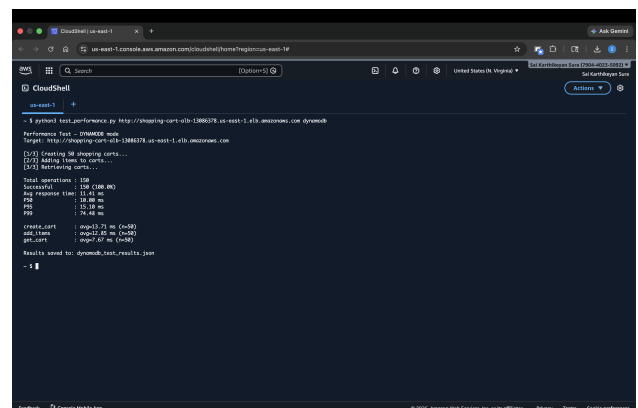
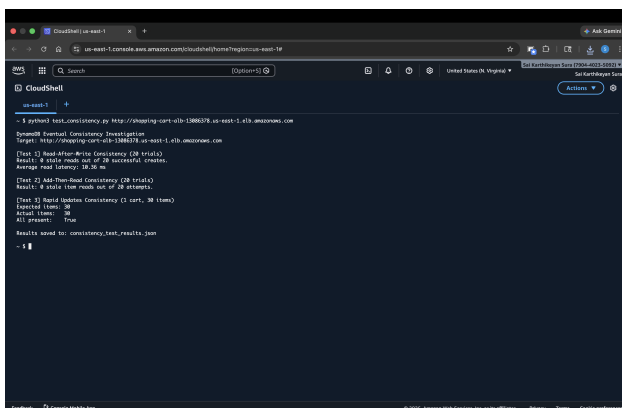


Figure 6: DynamoDB consistency and performance testing from CloudShell.

Key Challenges

Cold-start latency: The first DynamoDB request took 91.49ms, inflating the P99 to 74.48ms. Subsequent requests settled to 7–15ms. This appears to be TLS connection establishment to the DynamoDB API endpoint.

ID type mismatch bug (“String vs. Int”): After running Locust against DynamoDB, switching to MySQL mode caused a 26% failure rate. Locust workers retained UUID strings in memory from DynamoDB, but the MySQL backend expected integers (`strconv.Atoi` → 400 Bad Request). Multiple Locust reruns against MySQL continued failing because workers persisted stale cart IDs. Running `docker compose down` to fully wipe container state, then restarting Locust produced a clean run with 0% failures. This highlights a subtle integration testing pitfall: stateful load test clients can carry cross-backend contamination.

CloudWatch Monitoring

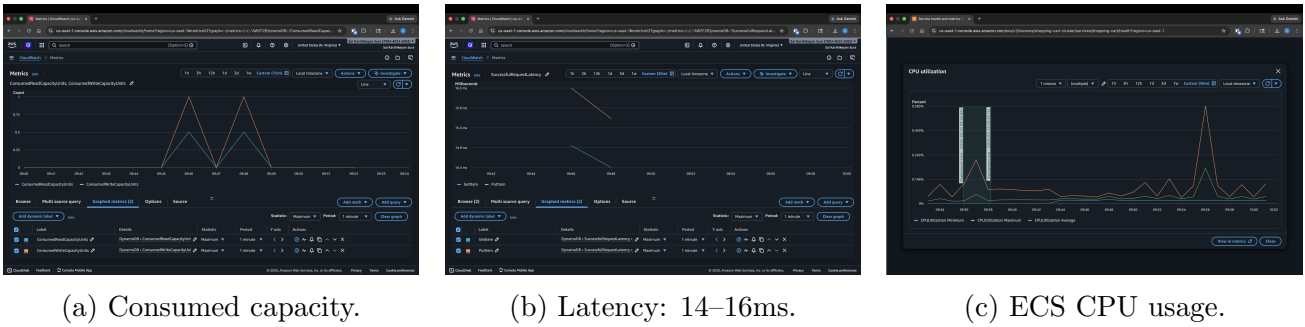


Figure 7: CloudWatch metrics during DynamoDB testing. No throttling events observed.

STEP III: Database Comparison & Analysis

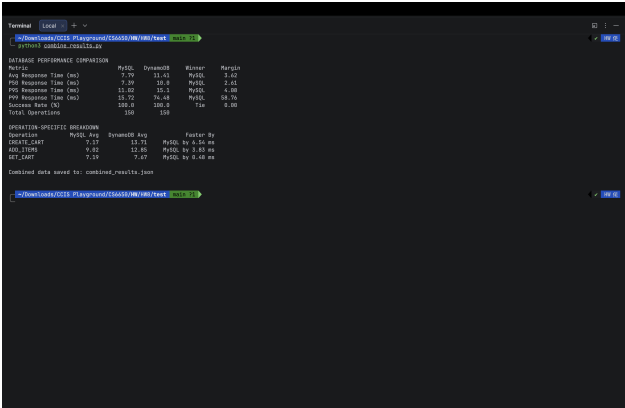


Figure 8: Output of `combine_results.py`: data source for all analysis below.

Part 1: Performance Comparison

Table 1: Overall Performance — Sequential Test (source: `combined_results.json`)

Metric	MySQL	DynamoDB	Winner	Margin
Avg Response Time (ms)	7.79	11.41	MySQL	3.62
P50 Response Time (ms)	7.39	10.00	MySQL	2.61
P95 Response Time (ms)	11.02	15.10	MySQL	4.08
P99 Response Time (ms)	15.72	74.48	MySQL	58.76
Success Rate (%)	100.0	100.0	Tie	0.00
Total Operations	150	150		

Table 2: Operation-Specific Breakdown — Sequential Test

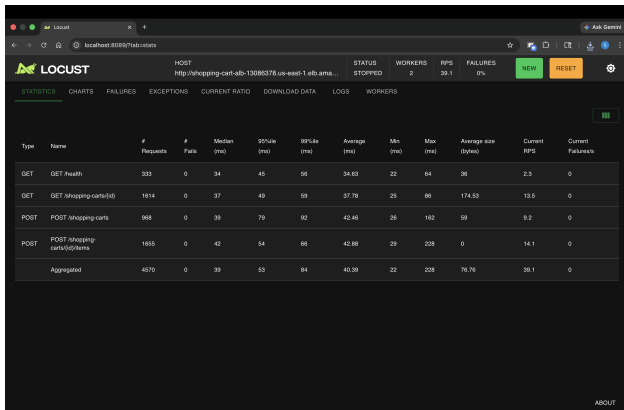
Operation	MySQL Avg (ms)	DynamoDB Avg (ms)	Faster By
CREATE_CART	7.17	13.71	MySQL by 6.54ms
ADD_ITEMS	9.02	12.85	MySQL by 3.83ms
GET_CART	7.19	7.67	MySQL by 0.48ms

Under Locust concurrent load (50 users, spawn rate 5), the gap narrowed significantly:

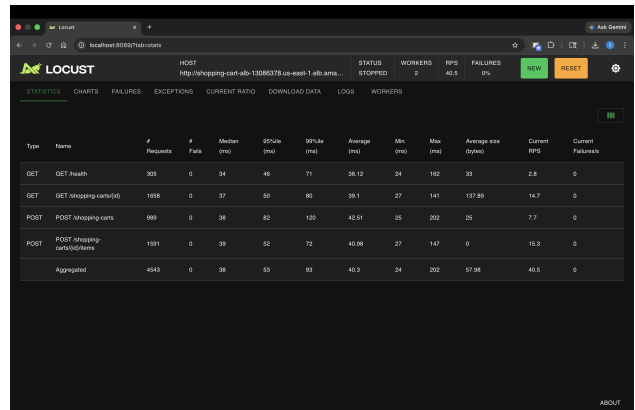
Table 3: Locust Load Test Comparison (50 users, 2 workers, clean runs)

Metric	MySQL	DynamoDB
Total Requests	4543	4570
Failures	0 (0%)	0 (0%)
Aggregated Avg (ms)	40.3	40.39
Aggregated Median (ms)	38	39
Aggregated P95 (ms)	53	53
Aggregated P99 (ms)	93	84
Max (ms)	202	228
RPS	40.5	39.1

Under concurrent load, the databases performed nearly identically. DynamoDB's P99 (84ms) was actually *better* than MySQL's (93ms), while MySQL had a lower max (202ms vs 228ms).

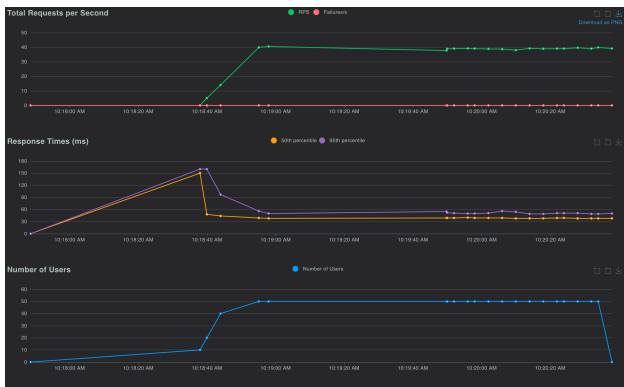


(a) DynamoDB: 4570 reqs, avg 40.39ms.

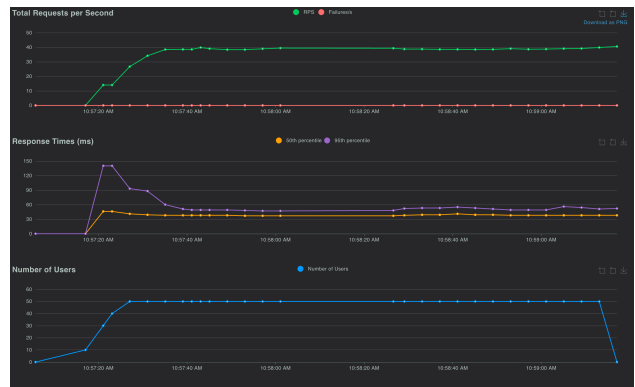


(b) MySQL: 4543 reqs, avg 40.3ms.

Figure 9: Locust statistics.



(a) DynamoDB: RPS and response time charts.



(b) MySQL: RPS and response time charts.

Figure 10: Initial ramp-up spike, then stable ~35–40 ms response time with steady throughput.

Consistency Model Impact: MySQL provides full ACID — every write is immediately and guaranteeably visible to subsequent reads. DynamoDB showed 0 stale reads in 60 trials because `GetItem` defaults to strongly consistent reads. Under high write contention, DynamoDB's

read-modify-write pattern for add-items could lose updates without conditional writes — a risk MySQL's atomic upsert avoids entirely. For shopping carts where a user expects immediate feedback after adding an item, both performed equivalently in our tests, but MySQL's guarantee is stronger by design. To design for DynamoDB's consistency model, applications should use optimistic locking via a version attribute and `ConditionExpression`, or leverage `TransactWriteItems` for multi-item atomicity.

Part 2: Resource Efficiency Analysis

Connection management: MySQL requires explicit pool tuning (`MaxOpenConns`, `MaxIdleConns`, `ConnMaxLifetime`). CloudWatch confirmed max 2 concurrent connections for 2 ECS tasks. DynamoDB eliminates this entirely — no pool, no connections, just stateless HTTP API calls.

Scaling: MySQL requires RDS instance class upgrades (vertical) or read replicas (horizontal reads only). RDS provisioning took 5m16s. DynamoDB with on-demand billing scales automatically with instant table creation. Under our Locust test, both sustained ~40 RPS with 0% failures, but DynamoDB's architecture would handle traffic spikes without pre-provisioning.

Predictability: In sequential tests, MySQL had a tighter latency distribution ($P99/P50 = 2.1\times$) vs DynamoDB ($P99/P50 = 7.4\times$). Under Locust load, both converged to similar distributions ($P99/P50 \approx 2.2\text{--}2.4\times$). DynamoDB's sequential tail latency was inflated by the cold-start spike (91.49ms first request).

Operational complexity: MySQL required retry logic for RDS readiness, connection pool tuning, schema migration code, and separate DDL statements. DynamoDB required AWS SDK setup, attribute marshaling, and read-modify-write logic. DynamoDB had fewer infrastructure moving parts but more application-level data management.

Part 3: Scenario Recommendations

Scenario A — Startup MVP (100 users/day, 1 dev, limited budget): **DynamoDB.** Zero operational overhead, no connection pool tuning, no RDS instance sizing. On-demand pricing means near-zero cost at low volume. Our 11.41ms avg sequential latency and 40.39ms under load are well within acceptable UX. A solo developer saves time not managing database infrastructure.

Scenario B — Growing Business (10K users/day, 5 devs, feature expansion): **MySQL.** As features grow beyond simple CRUD (order history joins, analytics queries, promotions with complex conditions), SQL expressiveness becomes essential. Our 7.79ms avg with tight P99 (15.72ms) provides predictable performance. The team can leverage SQL tooling (MySQL Workbench, query analyzers) for debugging.

Scenario C — High-Traffic Events (50K normal, 1M spike): **DynamoDB.** Auto-scaling handles $20\times$ traffic spikes without pre-provisioning. Under Locust load, DynamoDB achieved better P99 (84ms vs 93ms for MySQL). No throttling events in CloudWatch. MySQL would require over-provisioning a large RDS instance or adding read replicas for peak load.

Scenario D — Global Platform (millions of users, multi-region): **DynamoDB.** Global Tables provide multi-region replication with single-digit ms reads worldwide. MySQL multi-region requires manual replication setup and conflict resolution. For 24/7 global e-commerce, DynamoDB's managed global infrastructure is the practical choice.

Part 4: Architecture Recommendations

Shopping Cart Winner: Depends on workload pattern. **MySQL wins sequential tests** (3.62ms avg advantage, 58.76ms P99 advantage). **Both tie under concurrent load** (40.3ms vs 40.39ms avg). For predictable, steady traffic: MySQL. For spiky, auto-scaling needs: DynamoDB.

Supporting evidence: Sequential avg advantage: MySQL by 3.62ms. Sequential P99 advantage: MySQL by 58.76ms. Locust concurrent avg: MySQL 40.3ms vs DynamoDB 40.39ms (effectively tied). Locust P99: DynamoDB better (84ms vs 93ms). Success rate: 100% both.

When to choose the other: Choose DynamoDB over MySQL when auto-scaling is critical, multi-region is required, or the team lacks DBA expertise. Choose MySQL over DynamoDB when complex queries, strict ACID for multi-table operations, or relational data modeling is needed.

Polyglot Strategy for a complete e-commerce system: Shopping carts — DynamoDB (simple key-value access, needs auto-scaling for spikes). User sessions — DynamoDB (TTL expiration, high write volume, stateless lookups). Product catalog — MySQL (category filtering, full-text search, joins with inventory and pricing). Order history — MySQL (relational joins across orders/items/customers, analytical queries, ACID for financial records).

Part 5: Reflection

Empirical findings: (1) MySQL outperforming DynamoDB in sequential tests was unexpected — conventionally, DynamoDB is faster for simple key-value operations. The persistent connection pool eliminates per-request TCP/TLS overhead that DynamoDB incurs on every call. (2) Under concurrent Locust load, the performance gap completely vanished — both databases settled to nearly identical 40ms averages. This means the sequential test advantage is largely an artifact of connection reuse, not fundamental database speed. (3) The DynamoDB cold-start penalty (91.49ms first request) was surprisingly high.

What failed initially: (1) Schema migration crashed from multi-statement SQL in one `Exec()` call. (2) ECS tasks crash-looped before RDS was ready — added retry/backoff. (3) Docker layer caching prevented code deploys — required `terraform destroy` and reapply. (4) Locust ID type mismatch after switching from DynamoDB to MySQL caused persistent failures across multiple reruns. Only `docker compose down` (wiping all container state) followed by a fresh restart resolved it.

Key insights: I would choose MySQL when I need complex queries, tight sequential latency, and predictable steady-state traffic. I would choose DynamoDB when I need zero-ops, auto-scaling, and the access patterns are simple key-value. The single most important lesson: *test under realistic concurrent load, not just sequential operations*. The sequential MySQL advantage (3.62ms) disappeared entirely under Locust, revealing that real-world performance is dominated by network and concurrency factors, not raw database query speed. Hands-on implementation fundamentally changed my understanding: before this assignment, I assumed DynamoDB would always be faster for key-value workloads, but the data showed that connection pooling and network topology matter more than the database engine itself.

Considerations: Run performance tests from CloudShell, not your laptop. Use `docker compose down` between backend switches to avoid ID type contamination. Build retry logic into database initialization from day one. Test both sequential and concurrent — they tell very different stories.